Uppsala University

Scientific Computing for Data Analysis
VT2025

1TD352

# Miniproject 2: Classification of Handwritten digits

*Authors:*
Subash K C and Jakob Hjelm Redin

May 14, 2025

# 1 Introduction

Classification of handwritten digits is a standard problem in pattern recognition. A typical application is the automatic reading of zip codes on envelopes. This mini-project focuses on classifying handwritten digits (0-9) using Singular Value Decomposition (SVD). The goal of this mini-project is to develop a classification algorithm based on Singular Value Decomposition (SVD) to recognize handwritten digits. We are given, a set of hand-written digits, for which we know which digit is shown. Based on the training data, we will develop an algorithm, for classifying digits. Then we will test the classification algorithm on a separate test set which is also given. This report outlines the process of developing the algorithm, the results of the classification performance, and a discussion of the observed phenomena.

# 2 Approach and Method

## 2.1 Problem Description

We are given a dataset of handwritten digits, each represented as a 784-dimensional vector (28x28 pixels). The goal is to classify test images into one of ten digit categories (0 through 9) using Singular Value Decomposition (SVD).

Each digit $d$ is represented by a matrix $A_d$, where the columns span a linear subspace of $\mathbb{R}^{784}$. We will use SVD to decompose $A_d$ into orthogonal components:

$$A_d = U_d \Sigma_d V_d^T = \sum_{j=1}^{m} \sigma_j^{(d)} \mathbf{u}_j^{(d)} \mathbf{v}_j^{(d)T}$$

Here, $U_d$ contains the left singular vectors, which form an orthonormal basis for the subspace. Since the columns of $A_d$ are nearly linearly dependent, only a few leading columns of $U_d$ will capture the main features of the digit.

For a given test image $\delta$, we compute how well it can be represented by the singular vectors of each digit class. This is done by solving the least squares problem for each digit's subspace:

$$\min_x \|U_k x - \delta\|_2$$

## 2.2 Method

We will start by solving the least square problem and calculating the residual formula. Given the least squares problem:

$$\min_{\mathbf{x}} \|U_k\mathbf{x} - \boldsymbol{\delta}\|_2$$

where:

- $U_k \in \mathbb{R}^{m\times k}$ (first $k$ columns of $U$ from SVD)
- $\boldsymbol{\delta} \in \mathbb{R}^m$ (test image vector)
- $U_k$ has orthonormal columns ($U_k^T U_k = I_k$)

*Proof.* We will solve it using the normal equation. Let,

$$\begin{aligned}
\mathbf{x}^* &= \arg\min_{\mathbf{x}} \|U_k\mathbf{x} - \boldsymbol{\delta}\|_2^2 \\
&= \arg\min_{\mathbf{x}} (U_k\mathbf{x} - \boldsymbol{\delta})^T (U_k\mathbf{x} - \boldsymbol{\delta}) \\
&= \arg\min_{\mathbf{x}} \left(\mathbf{x}^T U_k^T U_k \mathbf{x} - 2\boldsymbol{\delta}^T U_k \mathbf{x} + \boldsymbol{\delta}^T\boldsymbol{\delta}\right) \\
&= \arg\min_{\mathbf{x}} \left(\mathbf{x}^T\mathbf{x} - 2\boldsymbol{\delta}^T U_k\mathbf{x} + \|\boldsymbol{\delta}\|_2^2\right) \qquad (\text{since } U_k^T U_k = I) \\
\nabla f(\mathbf{x}) &= 2\mathbf{x} - 2U_k^T\boldsymbol{\delta} = 0 \\
\Rightarrow \mathbf{x}^* &= U_k^T\boldsymbol{\delta}
\end{aligned} \tag{1}$$

The residual is:

$$\begin{aligned}
\|U_k\mathbf{x} - \boldsymbol{\delta}\|_2 &= \|U_k U_k^T\boldsymbol{\delta} - \boldsymbol{\delta}\|_2 \\
&= \|(U_k U_k^T - I)\boldsymbol{\delta}\|_2 \\
&= \|(I - U_k U_k^T)\boldsymbol{\delta}\|_2
\end{aligned}$$

□

To gain a deeper understanding of the behavior of the singular values and singular images, we will first compute the SVD and plot the first three singular images for digits 3 and 8 (each of size $784 \times 400$). Then we will implement SVD-based classification algorithm in a Python code, where only the first 400 training images are used for each digit. For each test image, we compute its residual relative to the subspaces of all 10 digits and classify the digit as the one with the smallest residual. This algorithm is then evaluated on the test set to verify its performance.

# 3 Implementation

The SVD-based classification algorithm was implemented in Python with the following steps:

1. **Training Phase:** For each digit $d \in \{0, 1, \ldots, 9\}$, we stack 400 training images into a matrix $A_d \in \mathbb{R}^{784\times 400}$, then we compute the SVD to obtain the first $k$ columns of $U_d$ as the subspace for classification.

   (a) We center the data by computing $\tilde{A}_d = A_d - \mu_d$, where $\mu_d$ is the mean image of digit $d$.

   (b) We compute the SVD: $\tilde{A}_d = U_d \Sigma_d V_d^T$.

   (c) Then we store the first $k$ columns of $U_d$ as $U_k^{(d)}$, along with $\mu_d$, for use during classification.

2. **Classification Phase:** For each test image $\delta$, we compute its residual relative to each digit's subspace and classify it as the digit with the smallest residual.

   (a) We center the test image: $\tilde{\delta}_d = \delta - \mu_d$ for each digit $d$.

   (b) We compute the residual for each class using:

   $$r_d = \left\| (I - U_k^{(d)} (U_k^{(d)})^T) \tilde{\delta}_d \right\|_2$$

   (c) Then we assign $\delta$ to the digit class $d$ with the smallest residual $r_d$.

# 4 Results

In this section, we begin by presenting the plots of singular values for digits 3 and 8. These plots help us analyze the distribution of singular values and provide insights into the dimensionality of the subspaces for each digit.

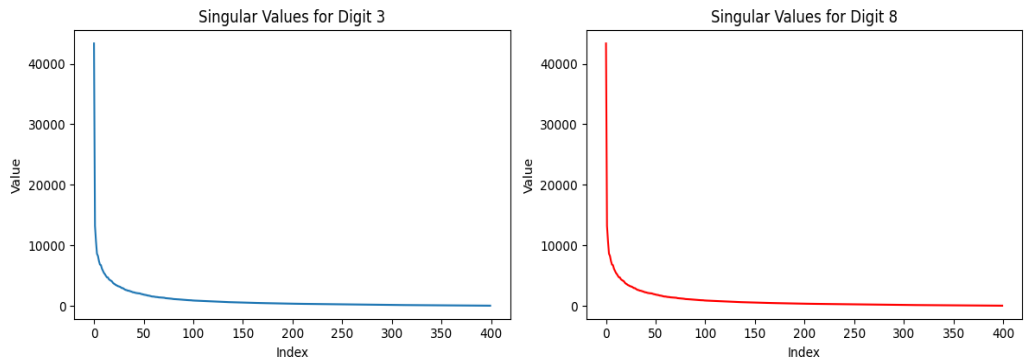


Figure 1: Singular Values for Digits 3 and 8

As shown in the plots1 above, the singular values of both digits exhibit a rapid decay. This indicates that most of the variance in the data can be captured by only a few leading singular vectors. The fast decay justifies the use of a low-rank approximation, as only the first $k$ singular vectors are needed to represent the essential structure of each digit.

Next, we proceed to visualize the first three singular images $u_1, u_2, u_3$ for digits 3 and 8. These images provide a deeper understanding of the fundamental components representing each digit.

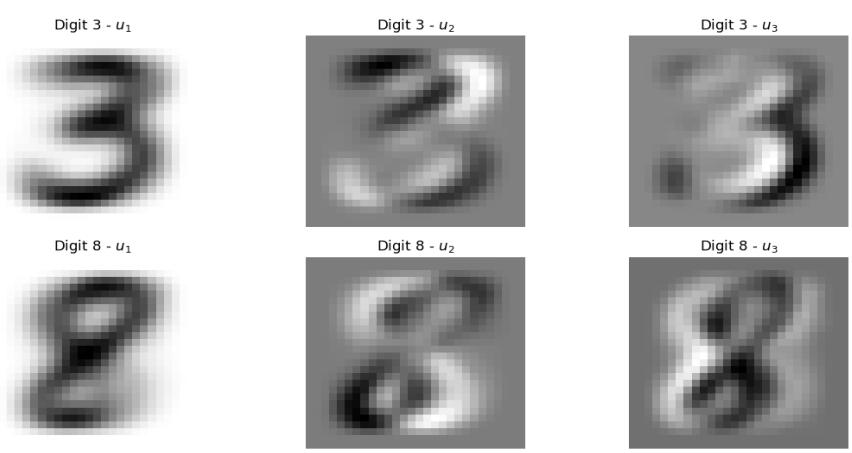


Figure 2: First three singular images for digits 3 and 8

These singular images2 are the principal components that capture the structure of the handwritten digits.

Next, we investigate how the number of singular vectors $k$ used in the subspace affects classification performance. The following plot3 shows the overall classification accuracy as a function of $k$, where $k$ ranges from 5 to 15.

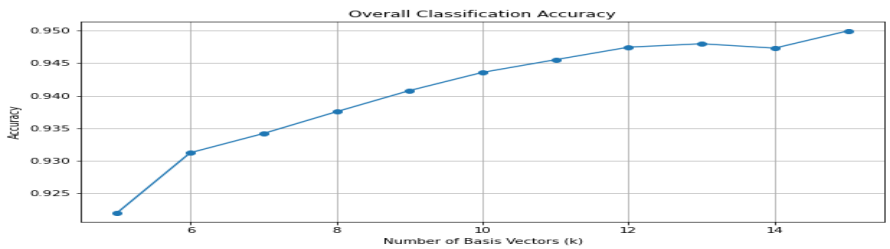


Figure 3: Classification accuracy as a function of the number of singular vectors $k$

As shown in Figure 3, the accuracy increases sharply with $k$ initially but the gain is minimal after a certain point (after $k = 12$) and even a small decrease from $k = 13$ to $k = 14$. But the highest precision is at $k = 15$, around 95% accuracy.

We also analyze the classification performance accuracy across individual digits(0 to 9). The following figure4 shows classification accuracy for each digit across the range of $k$ values.

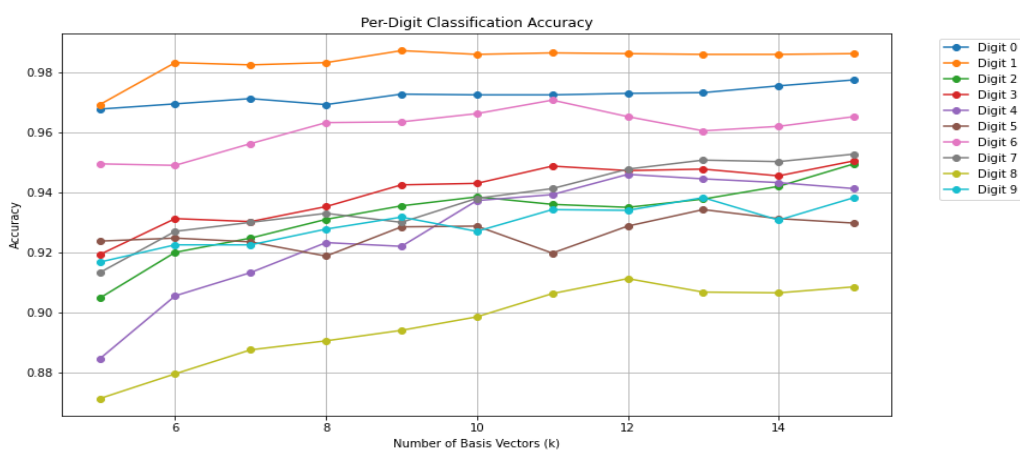


Figure 4: Classification accuracy per digit across k.

From Figure 4, we observe that some digits are classified with higher accuracy than others. This is likely due to the similarity in shape between certain digits, which leads to overlapping subspaces and increased misclassification.

| k | Overall Accuracy | Best Digit (Accuracy) | Worst Digit (Accuracy) |
|---|---|---|---|
| 5 | 0.9220 | 1 (0.9692) | 8 (0.8712) |
| 6 | 0.9312 | 1 (0.9832) | 8 (0.8795) |
| 7 | 0.9342 | 1 (0.9825) | 8 (0.8875) |
| 8 | 0.9375 | 1 (0.9832) | 8 (0.8905) |
| 9 | 0.9408 | 1 (0.9872) | 8 (0.8940) |
| 10 | 0.9436 | 1 (0.9860) | 8 (0.8985) |
| 11 | 0.9455 | 1 (0.9865) | 8 (0.9062) |
| 12 | 0.9475 | 1 (0.9862) | 8 (0.9113) |
| 13 | 0.9480 | 1 (0.9860) | 8 (0.9067) |
| 14 | 0.9473 | 1 (0.9860) | 8 (0.9065) |
| 15 | 0.9499 | 1 (0.9862) | 8 (0.9085) |

Table 1: Classification Accuracy Summary for $k = 5$ to 15

Table 1 summarizes the classification performance for values of $k$ ranging from 5 to 15. Digit 1 consistently achieves the highest accuracy across all $k$, while digit 8 is the most difficult to classify, likely due to its visual similarity with other digits.

# 5 Discussion

In this mini-project, Singular Value Decomposition (SVD) was applied to classify handwritten digits from a given dataset. The classification accuracy achieved using the SVD algorithm showed an improvement as the number of singular vectors(k=columns of U ) increased, as shown in Figure 3. As the number of singular vectors increased, the classification accuracy improved, with noticeable gains up to $k = 12$. After this point, improvements were marginal, suggesting that the majority of the important information for classification was captured within the first 12 singular vectors(columns of U). The observed rapid decay in singular values, as seen in Figure1, supports the idea that only a few singular vectors are essential for accurate classification.

However, the performance of the SVD-based classifier was not same across all digits. For example, digits such as 8 had accuracy of 91 percentage,

whereas the accuracy of digit 1 was 99 percentage for $k = 15$. One reason why digit 8 had much lower accuracy than digit 8 is because of the visual similarities with other digits. This also aligns with the theoretical fact that SVD performs well in linearly separable datasets, but struggles when there are significant overlaps in the feature space of certain classes. Despite these challenges, the SVD-based classifier achieved a competitive accuracy of 95% while remaining computationally efficient. Its performance can also be sensitive to noise and variations in digit shapes.

# 6 Conclusion

The SVD-based classifier worked surprisingly well — it showed that we could achieve high accuracy just by using the first few singular vectors(columns of $U$), which also kept the computation efficient. By projecting each digit onto its own subspace, the method was able to focus on the most important features needed to tell the digits apart. We found that using around 12 to 15 singular vectors struck a good balance between accuracy and complexity. However, due to the similarity between certain digits, classification was easier for digits like 1 compared to digits like 8 or 5, as shown in Figure4. To improve on this, we could explore other classification methods that handle noise better or do a better job of separating similar-looking digits.

# 7 References


1. Vilar, J. M. G., Kueh, H. Y., Barkai, N., & Leibler, S. (2002). Mechanisms of noise-resistance in genetic oscillators. *Proceedings of the National Academy of Sciences*, 99(9), 5988–5992.

2. Lectures notes

3. Mirzaei, D. (2023). *Numerical Linear Algebra: Least Squares, QR and SVD*. Lecture notes, Uppsala University.

4. Wikipedia.(2025). *Singular value decomposition*. Retrieved May, 2025, from https://en.wikipedia.org/wiki/Singular$_v alue_d ecomposition$

5. GeeksforGeeks.*Singular Value Decomposition (SVD)*. Retrieved May, 2025, from https://www.geeksforgeeks.org/singular-value-decomposition-svd/

# Appendix

# Appendix - Code

In this appendix, we have Python code used to perform data analysis and plot Figures for the project. All the required python code are order below

```python



# libraries
import numpy as np
import matplotlib.pyplot as plt


TrainMat = np.load("TrainDigits.npy")
TrainLab = np.load("TrainLabels.npy")

index = (TrainLab == 3); # find train digits of type 3
A3 = TrainMat[:,index[0]] # all train digits of type 3
A3 = A3[:,0:400] # the first 400 train digits of type 3

index = (TrainLab == 8)
A8 = TrainMat[:,index[0]]
A8 = A8[:,0:400]

# Step 5: Compute SVD

[tU3, tS3, tVt3] = np.linalg.svd(A3)


[U3, S3, Vt3] = np.linalg.svd(A3,full_matrices=False)


[tU8,tS8,tV8] = np.linalg.svd(A3)
[U8, S8, V8] = np.linalg.svd(A3,full_matrices=False)

D3 = np.reshape(tS3, (20, 20)).T # Reshaping a vector to a matrix
plt.imshow(D3, cmap ="gray")

D8 = np.reshape(tS8, (20, 20)).T # Reshaping a vector to a matrix
plt.imshow(D8, cmap ="gray")
plt.hist(tS8)
```

```python
print("t")
x = range(400)
np.array
print(len(x),x[1])



# Step 6: Plot singular values
plt.figure(figsize=(12, 4))


plt.subplot(1, 2, 1)
plt.plot(x,tS3)
plt.title("Singular Values for Digit 3")
plt.xlabel("Index")
plt.ylabel("Value")

plt.subplot(1, 2, 2)
plt.plot(x,tS8, color='red')
plt.title("Singular Values for Digit 8")
plt.xlabel("Index")
plt.ylabel("Value")

plt.tight_layout()
#plt.savefig('singularvaluesplot.png')
plt.show()



# to plot singular values for digits 3 and 8


def plot_singular_images(U3, U8):
    plt.figure(figsize=(12, 6))

    # Plot the first three singular images for digit 3
    for i in range(3):
        img = np.reshape(U3[:, i], (28, 28)).T
        plt.subplot(2, 3, i+1)  # Place the images in the first row
        plt.imshow(img, cmap='gray')
        plt.title(f'Digit 3 - $u_{i+1}$')
        plt.axis('off')

    # Plot the first three singular images for digit 8
```

```python
    for i in range(3):
        img = np.reshape(U8[:, i], (28, 28)).T
        plt.subplot(2, 3, i+4)  # Place the images in the second row
        plt.imshow(img, cmap='gray')
        plt.title(f'Digit 8 - $u_{i+1}$')
        plt.axis('off')

    plt.tight_layout()
    #plt.savefig('singularimages3and8.png')
    plt.show()


plot_singular_images(U3, U8)
```

Listing 1: Python Code for Task 2

```python


# Task 3:

# Libraries

import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
import seaborn as sns



# To load data
train_digits = np.load('TrainDigits.npy')
train_labels = np.load('TrainLabels.npy')
test_digits = np.load('TestDigits.npy')
test_labels = np.load('TestLabels.npy')


# To reshape the data to 1d
train_digits = np.load('TrainDigits.npy', allow_pickle=True).astype(np.float32)
train_labels = np.load('TrainLabels.npy', allow_pickle=True).astype(np.int8).flatten()  # Make 1D
test_digits = np.load('TestDigits.npy', allow_pickle=True).astype(np.float32)
test_labels = np.load('TestLabels.npy', allow_pickle=True).astype(np.int8).flatten()  # Make 1D
```

```python
# Prepare training data (first 400 images per digit)
digit_matrices = {}
for d in range(10):
    idx = np.where(train_labels == d)[0]
    digit_matrices[d] = train_digits[:, idx[:400]]  # Select first 400 columns



# Compute SVDs for all digits (store up to k=15)
svd_dict = {}
max_k = 15

for d in range(10):
    # Center the data
    mean_d = np.mean(digit_matrices[d], axis=1, keepdims=True)
    centered_data = digit_matrices[d] - mean_d

    # Compute SVD
    U, s, Vh = np.linalg.svd(centered_data, full_matrices=False)

    # Store components
    svd_dict[d] = {
        'U_k': U[:, :max_k],
        'mean': mean_d,
        'singular_values': s
    }



# Classification function for digits(vectorized)
def classify_digits(test_images, k):
    residuals = np.zeros((10, test_images.shape[1]))

    for d in range(10):
        U_k = svd_dict[d]['U_k'][:, :k]
        mean_d = svd_dict[d]['mean']

```

```python
        # Center test images and compute residuals
        centered = test_images - mean_d
        proj_residual = np.eye(784) - U_k @ U_k.T
        residuals[d] = np.linalg.norm(proj_residual @ centered, axis=0)

    return np.argmin(residuals, axis=0)  # Returns predictions (1D array)


# Evaluate for k=5 to 15
k_values = range(5, 16)
results = {
    'overall': [],
    'per_digit': {d: [] for d in range(10)}
}

for k in k_values:
    predictions = classify_digits(test_digits, k)

    # Overall accuracy
    results['overall'].append(np.mean(predictions == test_labels))

    # Per-digit accuracy
    for d in range(10):
        mask = (test_labels == d)
        if mask.any():
            digit_predictions = predictions[mask]
            accuracy = np.mean(digit_predictions == d)
            results['per_digit'][d].append(accuracy)



# Plot overall accuracy
plt.figure(figsize=(10,5))
plt.plot(k_values, results['overall'], 'o-')
plt.xlabel('Number of Basis Vectors (k)')
plt.ylabel('Accuracy')
plt.title('Overall Classification Accuracy')
plt.grid(True)
#plt.savefig('overallaccuracy.png')
plt.show()

# Plot per-digit accuracy
```

```python
plt.figure(figsize=(12,6))
for d in range(10):
    plt.plot(k_values, results['per_digit'][d], 'o-', label=f'Digit {d}')
plt.xlabel('Number of Basis Vectors (k)')
plt.ylabel('Accuracy')
plt.title('Per-Digit Classification Accuracy')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True)
plt.tight_layout()
#plt.savefig('perdigitaccuracy.png')
plt.show()

# Confusion matrix for best k
best_k = k_values[np.argmax(results['overall'])]
predictions = classify_digits(test_digits, best_k)
cm = confusion_matrix(test_labels, predictions)

plt.figure(figsize=(10,8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=range(10), yticklabels=range(10))
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title(f'Confusion Matrix (k={best_k}, Accuracy={np.max(results["overall"]):.4f})')
#plt.savefig('confusionmatrix_k15.png')
plt.show()

# Print numerical results
print("Classification Results:")
print("k\tOverall\tBest Digit\tWorst Digit")
print("-"*50)
for i, k in enumerate(k_values):
    current_accs = [results['per_digit'][d][i] for d in range(10)]
    best_d = np.argmax(current_accs)
    worst_d = np.argmin(current_accs)
    print(f"{k}\t{results['overall'][i]:.4f}\t"
          f"{best_d}({current_accs[best_d]:.4f})\t"
          f"{worst_d}({current_accs[worst_d]:.4f})")
```

Listing 2: Python Code for Task 3- SVD-based classification algorithm

# Appendix - Figures and Plots

This section includes all the figures and plots not included within the main sections.

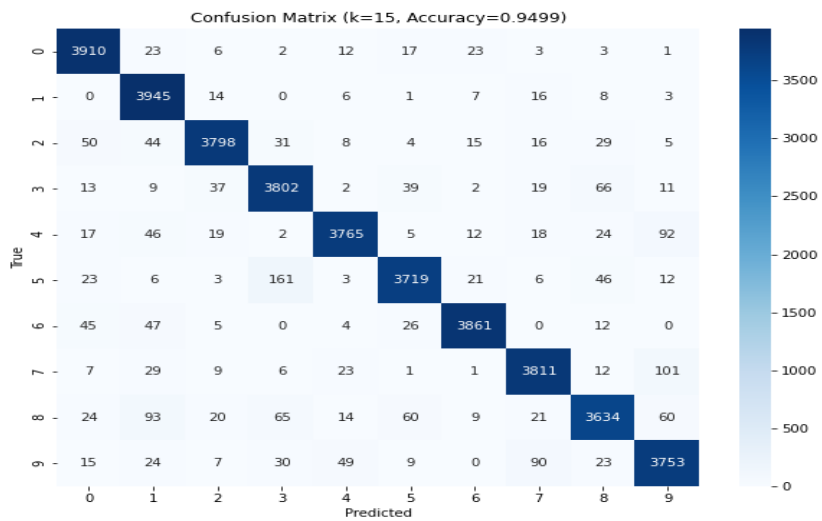


Figure 5: Classification accuracy matrix.